

En första klass

Allmänt definieras en enkel klass i C++ på följande sätt:

```
class namn {  
    metoddefinitioner;  
    variabeldefinitioner;  
};
```

Det viktigaste i en klassdefinition är nyckelordet `class` och `{}`. Mera behövs inte för att definiera en minimal klass som inte innehåller någonting alls. Vi skall nu skapa en första konkret klass i C++. Det är en enkel klass som abstraherar en punkt i en tvådimensionell värld. I många sammanhang behövs en dylik klass och vi kommer att använda den i ett senare exempel. Klassen `Coordinate` skall innehålla två *medlemmar* som representerar x- och y-koordinaterna.

```
class Coordinate {  
public:  
    // konstruktor  
    Coordinate (float X, float Y) { m_X = X; m_Y = Y; };  
  
    // medlemmar  
    float m_X;  
    float m_Y;  
};
```

Detta är en fungerande lösning men inte en optimal. Vi har här en *konstruktor* som används för att initialisera ett objekt av typen `Coordinate`, samt två medlemmar som innehåller den data vi vill abstrahera, nämligen två värden. Den kod som körs i konstruktorn (mera om det i [avsnittet Konstruktor och destruktor](#)) är skriven direkt efter definitionen. Detta är möjligt, och vi får då en Java-liknande syntax, men vi skall senare se hur det är möjligt att flytta ut koden som hör till metoder till skilda platser. För enkla och korta metoder duger denna approach dock fint. Flaggan `public:` betyder att alla definitioner som kommer efter flaggan är öppna för vem som helst att läsa och skriva. Det betyder att våra medlemmar `x` och `y` kan accesseras av vem som helst. Denna enkla klass kan *instantieras* till objekt i ett program på följande sätt:

```
Coordinate C = Coordinate ( 3, 5 );
```

Detta är hittills det enda sätt på vilket vi kan initialisera en instans av klassen `Coordinate`. Vi skall senare visa hur man kan skapa andra konstruktorer för detta syfte. För att sedan använda klassen används samma syntax som för `struct`, d.v.s. man accesserar medlemmar och metoder via en punkt (`.`). För att använda vår nya klass kunde vi göra följande:

```
Coordinate C = Coordinate ( 3, 5 );  
cout << "Koordinaterna är " << C.m_X << ", " << C.m_Y << endl;  
C.X = 3.3;  
C.Y = -4.8;
```

De används alltså precis som en `struct`. En klass kan i princip ses på som en utvidgad `struct`. Vi har använt oss av prefixet `m_` för att markera våra medlemmar. Det är inte nödvändigt, men gör det lättare att skilja på medlemmar och parametrar, såsom i konstruktorn. Man kan även använda denna

kortare definition för instantiering:

```
Coordinate C ( 3, 5 );  
...
```

Där ges parametrarna direkt efter variabelnamnet. Man kan i dessa enkla exempel använda vilken metod som helst, men det finns några argument (som vi tar upp senare) som förespråkar att man använder den senare metoden.

Dataskydd

Med *dataskydd* avses här möjligheter för klasser att skydda intern data för utomstående. Klassen ovan har inget dataskydd överhuvudtaget. Vi vill att vår klass kan kontrollera hur dess data accesseras. Vi skall nu introducera en ny dataskyddsflagga som kallas *private*: i vår klass:

```
class Coordinate {  
public:  
    // konstruktor  
    Coordinate (float X, float Y) { m_X = X; m_Y = Y; };  
  
private:  
    // medlemmar  
    float m_X;  
    float m_Y;  
};
```

Vi har nu placerat våra medlemmar som *privata* medlemmar. Detta betyder att ingen utomstående kan accessera dem på något sätt. Endast klassen själv kan accessera dem. Hur ska vi nu komma åt dem då? Vi kan introducera några nya metoder för att både läsa och skriva våra värden:

```
class Coordinate {  
public:  
    // konstruktor  
    Coordinate (float X, float Y) { m_X = X; m_Y = Y; };  
  
    // accessera medlemmar  
    float x () { return m_X; };  
    float y () { return m_Y; };  
    void setX (float X) { m_X = X; };  
    void setY (float Y) { m_Y = Y; };  
  
private:  
    // medlemmar  
    float m_X;  
    float m_Y;  
};
```

Vi har nu ett väldefinierat gränssnitt som används för att accessera våra datamedlemmar. Vi måste nu ändra vårt textprogram ovan till följande:

```
Coordinate C = Coordinate ( 3, 5 );  
cout << "Koordinaterna är " << C.x() << ", " << C.y() << endl;  
C.setX ( 3.3 );  
C.setY ( -4.8 );
```

Notera att accessmetoderna är *public* medan själva data ännu är *private*. På så sätt kan utomstående anropa metoderna, men inte accessera data direkt. Det finns en del nytta med ett dylikt

förfarande, bl.a. har vi *datasäkerhet* och *abstraktion*. Användare tvingas accessera objektet via ett väldefinierat gränssnitt som fungerar så att objektet aldrig blir inkonsistent. I detta exempel är det ingen skillnad på vilka värde som ges in, men man kan i dessa metoder t.ex. placera in kontroller för att given data är korrekt.

Man kan i C++ explicit markera att en viss metod inte modifierar ett objekt då den anropas. Detta behövs då man har objekt som är definierade som `const`, och som då inte får anropa metoder som skulle ändra dessa. Vi ser ju enkelt at metoden `x()` inte ändrar objektet för vilket metoden anropas, men vi kan ändå inte anropa metoden för ett `const` objekt. Lösningen blir då att explicit specificera i klassdefinitionen vilka metoder som inte ändrar på ett objekt. I vårt fall är det emtoderna `x()` och `y()`. De övriga ändrar ju någonting. Detta görs genom att sätta en definition `const` efter parameterdefinitionerna till en metod enligt:

```
...
// accessera medlemmar
float x () const { return m_X; };
float y () const { return m_Y; };
...
```

Nu kan dessa metoder bättre användas med `const` objekt.

[Förutgående](#)
Klasser

[Hem](#)
[Upp](#)

[Nästa](#)
Konstruktor och destruktor